

Blockchain-Based Electronic Health Record System

Purpose

This document proposes the **Agentic AI architecture** for the EHR project.

The objective is not to replace the existing Gemini-based AI implementation, but to evolve it into a **controlled multi-agent system** capable of:

- understanding clinical tasks,
- selecting appropriate agents,
- retrieving authorized clinical context,
- using controlled tools,
- producing assistive clinical insights,
- maintaining execution state,
- supporting human review,
- and recording AI provenance.

Security, FHIR standardization, blockchain infrastructure and dashboard work are treated as **cross-team dependencies**, not as the primary responsibility of the Agentic AI team.

Index

1. Current System Analysis
2. Problem / Gap Analysis
3. Requirements
4. Proposed Agentic AI Architecture
5. Technology Choices
6. Feature Specification
7. Integration With Existing Project
8. Data Model
9. Security & Cross-Team Dependencies
10. Failure Handling
11. Scalability & Future Work
12. Implementation Plan
13. Testing Plan

14. Demo Plan

15. Open Questions

1. Current System Analysis

1.1 What exists today?

The current project already contains a Gemini-based AI pipeline with two AI components:

```
Lab Report
  ↓
Report Extraction
  ↓
Lab Report Analysis Agent
  ↓
Clinical Decision Support Agent
  ↓
AI Result
```

The existing system supports:

- laboratory report analysis,
- abnormality detection,
- clinical decision-support suggestions,
- patient-history retrieval,
- Gemini-based generation,
- IPFS storage of AI output,
- Fabric metadata anchoring,
- fallback heuristic analysis when Gemini is unavailable.

The project documentation confirms that the Gemini AI workflow is already integrated into the Lab Gateway.

1.2 Existing components

The main AI-related implementation currently exists around:

```
orgs/lab/EHR-LABORG-main/  
└─ client/  
    └─ node-gateway/  
        ├── ai-agent.js  
        ├── server.js  
        ├── gateway.js  
        ├── report-extractor.js  
        └─ ipfs.js
```

The main AI logic is currently in:

```
ai-agent.js
```

The API/orchestration entry point is:

```
server.js
```

The Fabric interaction is handled through:

```
gateway.js
```

and report processing through:

```
report-extractor.js
```

The existing gateway exposes `/api/ai/analyze` and `/api/records/ipfs` .

1.3 Existing agents

Lab Report Analysis Agent

Current responsibility:

```
Input  
↓  
Lab result / report text  
↓  
Gemini  
↓  
Summary
```

```
Abnormalities
Risk flags
Confidence
```

The current implementation already explicitly defines this as an assistive, non-decision-making component.

Clinical Decision Support Agent

Current responsibility:

```
Current result
+
Patient history
↓
Gemini
↓
Recommendation
Possible risks
Next steps
Confidence
```

It is also explicitly designed to remain non-diagnostic.

1.4 Existing fallback

If Gemini is unavailable:

```
Gemini
↓
Failure
↓
Fallback heuristic analysis
```

The current implementation returns low-confidence heuristic results instead of completely stopping the workflow.

This should be retained.

1.5 What should NOT be rewritten?

Keep

- Gemini integration
- `runLabReportAnalysisAgent()`
- `runClinicalDecisionSupportAgent()`
- `report-extractor.js`
- existing `/api/ai/analyze`
- existing `/api/records/ipfs`
- existing IPFS AI artifact storage
- existing fallback mechanism

Refactor

```
Current ai-agent.js
      ↓
Separate agent modules
      ↓
Orchestrator
      ↓
Tool layer
```

The goal is **refactoring into an agent architecture**, not rewriting existing functionality.

2. Problem / Gap Analysis

2.1 Current architecture is mostly a fixed pipeline

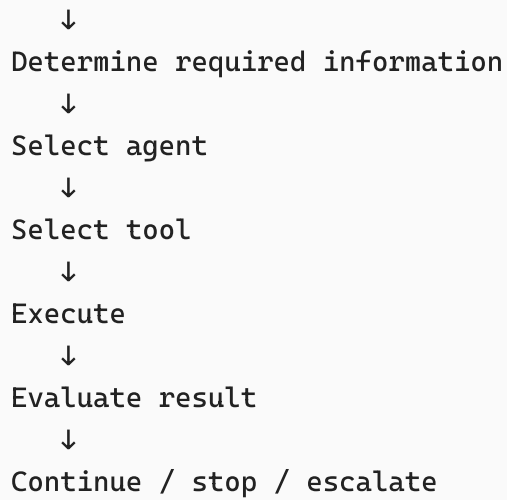
Current:

```
Lab Agent
      ↓
Clinical Agent
```

The sequence is largely predetermined.

A true agentic workflow should support:

```
Request
      ↓
Understand task
```



2.2 No dedicated orchestrator

The system currently has AI functions but no independent orchestration layer responsible for deciding:

- which agent should run,
- which context is required,
- which tools are needed,
- when another agent should be called,
- when human review is required.

2.3 No controlled tool layer

Agents currently receive information through the gateway.

The proposed architecture needs:



The LLM should **never directly control Fabric, IPFS or databases**.

Gemini function calling is suitable for this model because the model requests a function while the application executes the actual function. ([Gemini function calling](#))

2.4 No explicit agent state

A multi-agent workflow needs to remember:

```
Current task
Completed agents
Agent outputs
Tool results
Errors
Pending approval
```

Example:

```
runId = RUN-001

Lab Agent      ✓
History Agent  ✓
Clinical Agent ✓
Governance     pending
```

2.5 No formal agent identity and capability model

We need to define:

```
Agent
├─ Role
├─ Version
├─ Allowed tools
├─ Allowed data
└─ Organization
```

This is primarily an **AI + Security Team responsibility**.

2.6 No dedicated governance stage

Current safety is largely prompt-based.

The proposed architecture introduces:

```
graph TD; A[AI Output] --> B[Governance]; B --> C[Human Review];
```

AI Output
↓
Governance
↓
Human Review

This allows deterministic validation outside the LLM.

2.7 Main risks

The Agentic AI team should specifically consider:

- hallucination,
- wrong tool selection,
- prompt injection,
- incorrect patient context,
- excessive agent autonomy,
- incomplete context,
- misleading confidence,
- agent failure,
- provenance gaps.

Security-specific threats such as authentication, encryption and network security should be handled by the **Security Team**.

3. Requirements

3.1 Functional Requirements

FR-01 — Agent orchestration

The system must support multi-step agent workflows.

FR-02 — Specialized agents

Initial agents:

1. Lab Report Agent
2. Patient History Agent
3. Clinical Decision Support Agent
4. Governance Agent

Future:

5. FHIR Agent
6. Pharmacy Agent
7. Multimodal Agent

FR-03 — Tool calling

Agents must interact with external systems through controlled tools.

FR-04 — Workflow state

Every agent execution should have a unique `runId` .

FR-05 — Human review

Clinical recommendations must support human review.

FR-06 — AI provenance

Important AI executions should produce provenance information.

FR-07 — Agent observability

Agent execution should be traceable through logs/events.

3.2 Non-functional Requirements

The AI architecture should be:

- modular,
- maintainable,
- observable,

- testable,
- fault tolerant,
- framework-independent at the architecture level.

Large-scale optimization is **Future Work**.

3.3 Security Requirements

AI-specific:

- tool permissions,
- prompt-injection protection,
- output validation,
- agent isolation,
- PHI minimization,
- provenance.

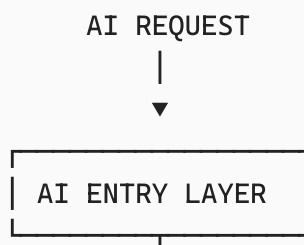
General security:

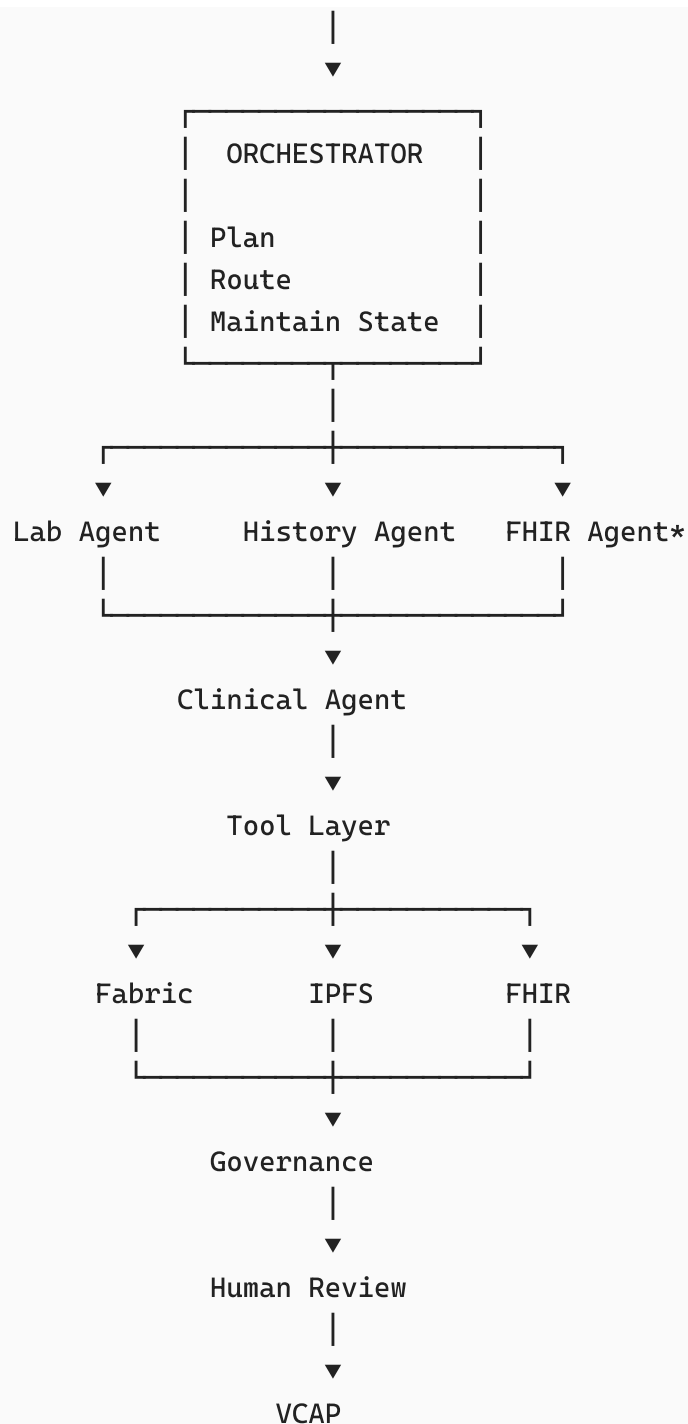
- authentication,
- Fabric identity,
- ABAC,
- encryption,
- network security,
- secrets management.

Owner: Security Team

4. Proposed Agentic AI Architecture

4.1 High-Level Architecture





* FHIR Agent is a future/parallel dependency and should be coordinated with the **FHIR Team**.

4.2 Agent Orchestrator

The orchestrator is the central component of the proposed architecture.

Responsibilities:

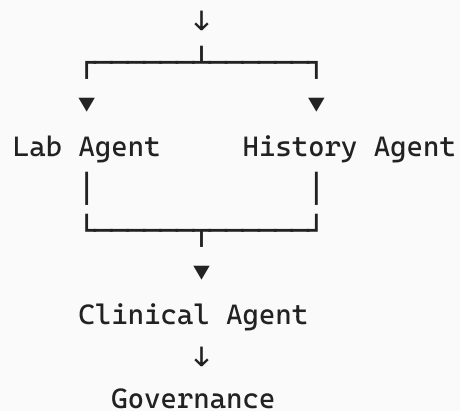
```
Receive request
  ↓
Understand task
  ↓
Determine required context
  ↓
Select agents
  ↓
Execute workflow
  ↓
Track state
  ↓
Handle failures
  ↓
Send result to governance
```

Example:

```
User:
"Analyze this patient's latest CBC with historical trends."
```

↓

Orchestrator



4.3 Agent 1 — Lab Report Agent

Responsibility

Analyze the current laboratory report.

Input

```
PDF/image extracted text
resultData
testCode
```

Output

```
{
  "summary": "...",
  "abnormalities": [],
  "riskFlags": [],
  "confidence": "medium"
}
```

This directly evolves the existing Lab Report Analysis Agent.

4.4 Agent 2 — Patient History Agent

Responsibility

Retrieve and summarize relevant historical information.

```
Patient ID
  ↓
Retrieve relevant records
  ↓
Build timeline
  ↓
Identify trends
```

Example:

```
Hb:

Jan  13.2
Mar  12.1
Jun  10.4
Sep   8.5
```

Trend → decreasing

The current gateway already retrieves patient history through `getLabResultsByPatient()`, so this functionality can be moved behind the future History Agent/tool boundary rather than rewritten.

4.5 Agent 3 — Clinical Decision Support Agent

Responsibility

Combine:

```
Current findings
+
Historical context
+
Clinical context
```

and produce:

```
Possible risks
Suggested review steps
Next steps
Confidence
```

The existing implementation already performs this role and should be refactored rather than recreated.

4.6 Agent 4 — Governance Agent

Responsibility

Review the generated AI output.

Checks:

```
Is output advisory?
Is uncertainty expressed?
```

Were allowed tools used?
Is required context available?
Is provenance available?
Does the result require human review?

Output:

APPROVED
REJECTED
HUMAN_REVIEW_REQUIRED

This should be implemented as a **hybrid of deterministic policies and AI-based review**, with security-critical decisions remaining deterministic.

4.7 Future Agents

These should **not be part of the initial implementation**.

FHIR Agent

Responsible for:

Custom data
↓
FHIR mapping
↓
FHIR validation

Dependency/Owner: FHIR Team

Pharmacy Agent

Responsible for medication-related assistance.

Dependency/Owner: Pharmacy/Security Team

Multimodal Agent

Responsible for:

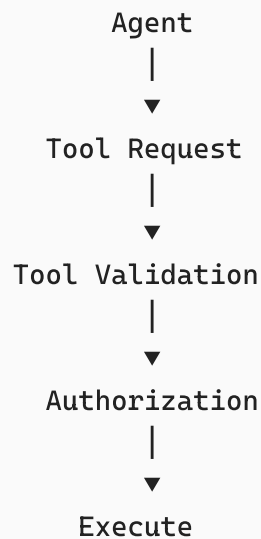
X-ray
MRI

Future Work

The current roadmap already identifies multimodal AI as a later extension.

4.8 Tool Layer

The tool layer separates reasoning from system actions.



Initial tools:

```
getPatientHistory()  
getLabResults()  
getPatient()  
getDocument()  
checkConsent()
```

Future tools:

```
getFHIRResource()  
createFHIRResource()  
createVCAP()  
verifyProvenance()  
requestHumanApproval()
```

4.9 Agent State

Every workflow should maintain state.

Example:

```
{
  "runId": "RUN-001",
  "task": "CBC_ANALYSIS",
  "patientId": "PAT-001",

  "completedAgents": [
    "lab-agent",
    "history-agent"
  ],

  "pendingAgent": "clinical-agent",

  "status": "RUNNING"
}
```

The state should allow the workflow to resume or fail gracefully.

4.10 Complete Agent Flow

```
Lab Report
  ↓
AI Request
  ↓
Orchestrator
  ↓
Lab Agent
  ↓
History Agent
  ↓
Clinical Agent
  ↓
Governance
  ↓
Human Review
  ↓
VCAP
```

5. Technology Choices

Technology	Decision	Purpose
Gemini	Keep	LLM
Node.js	Keep	AI service/tool layer
Express	Keep	API layer
LangGraph	Recommended	Agent orchestration/state
Hyperledger Fabric	Keep	Trusted ledger
IPFS	Keep	AI/clinical artifacts
FHIR	Add	Standard clinical representation
VCAP	Extend	AI provenance

5.1 LangGraph

Recommended for the orchestration layer because the workflow naturally forms a graph:



LangGraph supports stateful graph workflows, agent handoffs and human-in-the-loop patterns.

However:

The architecture should not depend conceptually on LangGraph.

A custom orchestrator should remain a possible alternative.

5.2 Alternatives

Custom Node.js orchestrator

Pros:

- simple
- lightweight
- no additional framework

Cons:

- state management becomes custom
- complex workflows become harder

CrewAI

Useful for multi-agent systems, but adds another framework into the existing stack.

Direct Gemini calls

Suitable for the existing simple workflow but insufficient for the proposed stateful multi-agent architecture.

6. Feature Specification

Feature 1 — Agent Orchestration

```
Request
↓
Orchestrator
↓
Agent selection
↓
Execution
```

Feature 2 — Patient History Agent

```
Patient
↓
Authorized history
↓
Relevant timeline
↓
Trend
```

Feature 3 — Tool Calling

Agents can request controlled tools.

Example:

```
Clinical Agent
↓
getPatientHistory()
↓
Authorization
↓
Fabric
```

Feature 4 — Agent State

Track:

```
runId
agent
status
outputs
errors
tool calls
```

Feature 5 — Governance

Validate:

```
Output
Tool usage
Confidence
Required context
Policy
```

Feature 6 — Human Review

```
AI Result
↓
Doctor
├─ Approve
├─ Modify
└─ Reject
```

Feature 7 — AI Provenance

VCAP should eventually capture:

```
Agent ID
Agent version
Model
Prompt hash
Input hash
Tool calls
Output hash
Timestamp
Human review
```

The existing VCAP work already proposes patient-record hash, prompt hash, model fingerprint, summary hash, timestamp and gateway signer. The additional agent/tool fields should be treated as an extension.

7. Integration With Existing Project

7.1 Files to modify

Primary:

```
client/node-gateway/ai-agent.js  
client/node-gateway/server.js
```

Secondary:

```
client/node-gateway/gateway.js  
client/node-gateway/ipfs.js  
client/node-gateway/report-extractor.js
```

7.2 New structure

```
node-gateway/  
|  
├── agents/  
|   ├── lab-agent.js  
|   ├── history-agent.js  
|   ├── clinical-agent.js  
|   └── governance-agent.js  
|  
├── orchestrator/  
|   ├── orchestrator.js  
|   └── state.js  
|  
├── tools/  
|   ├── fabric-tools.js  
|   ├── ipfs-tools.js  
|   └── consent-tools.js  
|  
└── provenance/  
    └── vcap.js
```

7.3 Existing API compatibility

Keep:

```
POST /api/ai/analyze
POST /api/records/ipfs
```

Potential future API:

```
POST /api/agents/run
GET /api/agents/:runId
GET /api/agents/:runId/status
POST /api/agents/:runId/approve
POST /api/agents/:runId/reject
```

7.4 Chaincode

No major chaincode rewrite is required for the first Agentic AI phase.

AI should initially continue using:

```
Fabric query
+
existing lab-result metadata
```

VCAP-specific chaincode changes should be coordinated with the **Blockchain/Security Team**.

7.5 Database

No major database change is required initially.

Agent workflow state can initially remain lightweight.

A dedicated scalable state store is **Future Work**.

8. Data Model

8.1 Agent Run

```
{
  "runId": "RUN-001",
```

```
"patientId": "PAT-001",  
"task": "LAB_ANALYSIS",  
"status": "COMPLETED",  
"startedAt": "...",  
"completedAt": "..."  
}
```

8.2 Agent Output

```
{  
  "agentId": "lab-agent",  
  "agentVersion": "1.0",  
  "output": {},  
  "confidence": "medium"  
}
```

8.3 Tool Call

```
{  
  "tool": "getPatientHistory",  
  "agentId": "clinical-agent",  
  "timestamp": "...",  
  "resultHash": "..."  
}
```

8.4 AI Artifact

Large AI output:

IPFS

Reference:

Fabric

This follows the existing project approach where the AI summary is stored in IPFS and its CID/digest are included in on-chain AI metadata.

9. Security & Cross-Team Dependencies

Security is **not the primary Agentic AI team's responsibility**, but the AI architecture must expose clear requirements to the Security Team.

Security Team owns

- Authentication
- Fabric identity
- ABAC/RBAC
- Consent enforcement
- Encryption
- Key management
- TLS
- Secrets
- Network security
- Audit infrastructure

Agentic AI team owns

- Agent permissions model
- Tool permission requirements
- Prompt injection protection
- Agent isolation
- AI output validation
- AI safety rules
- Provenance requirements
- Human-review requirements

Shared responsibility

- Agent Identity
- Tool Authorization
- Patient-data access

10. Failure Handling

Only core failure handling is part of the initial implementation.

AI failure

```
Gemini fails
  ↓
Fallback heuristic
  ↓
Low confidence
  ↓
Human review
```

This already exists and should be retained.

Agent failure

```
Agent fails
  ↓
Record failure
  ↓
Retry if appropriate
  ↓
Continue only if required context exists
```

Tool failure

```
Tool fails
  ↓
Do not fabricate result
  ↓
Mark context unavailable
```



Escalate / retry

Governance failure

Governance fails



Do NOT approve result



Human review

11. Scalability & Future Work

Scalability should **not drive the initial implementation**.

The first goal is a correct and demonstrable agent architecture.

Future Work

Large-scale patient data

10K → 100K → 1M+ records

Future requirements:

- indexed retrieval,
- pagination,
- context compression,
- caching.

Agent worker scaling

Future:

Orchestrator



```
Job Queue
  ↓
Agent Worker Pool
```

Distributed agents

Current proposal:

```
Logical Agent
  ↓
Deployable Service
```

Future deployment:

```
Machine 1 → Lab Agent
Machine 2 → Clinical Agent
Machine 3 → Governance
```

The existing project documentation already describes this three-machine division as a possible production-style split.

For the initial implementation, however, **logical separation is more important than physical separation**.

Future agents

- Pharmacy Agent
 - Multimodal Agent
 - Advanced FHIR Agent
 - Patient longitudinal reasoning agent
 - Research/knowledge agent
-

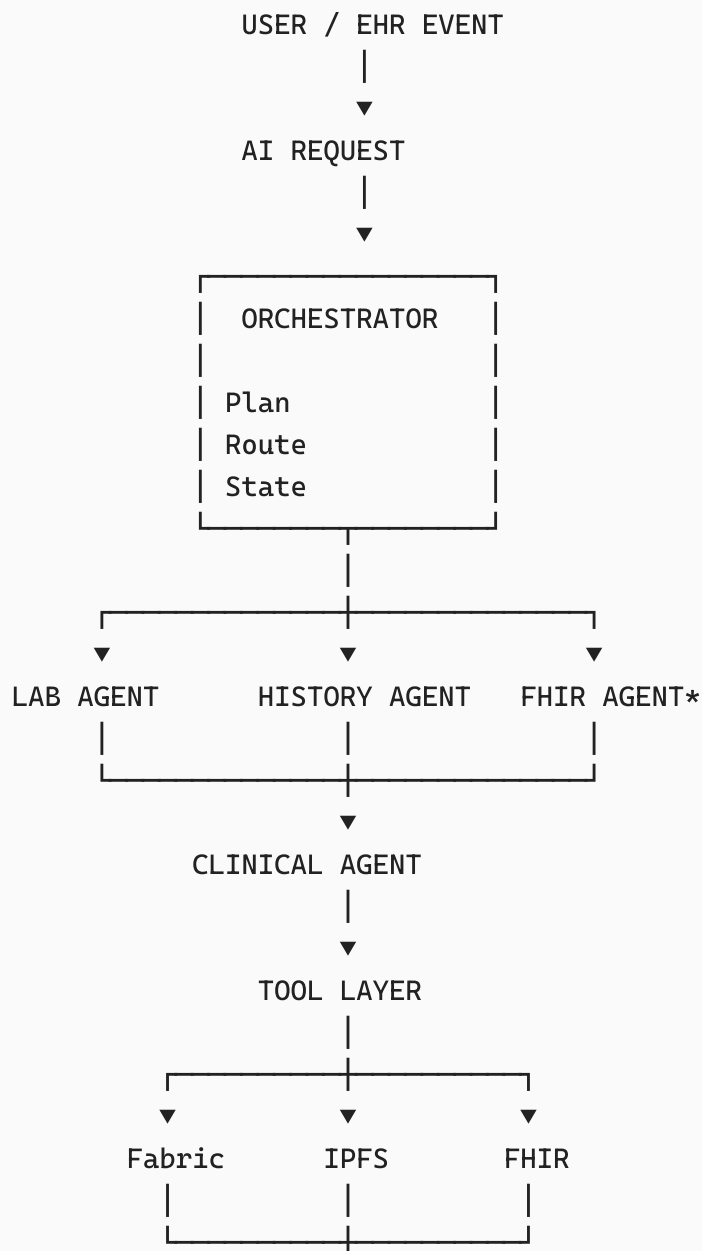
Future capabilities

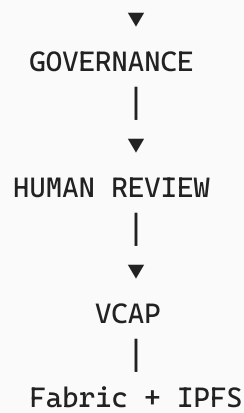
- asynchronous workflows,
- event-driven agents,

- agent-to-agent communication,
- advanced memory,
- distributed deployment,
- model routing,
- model evaluation,
- cost optimization.

Final Architecture

The Agentic AI pillar should ultimately be:



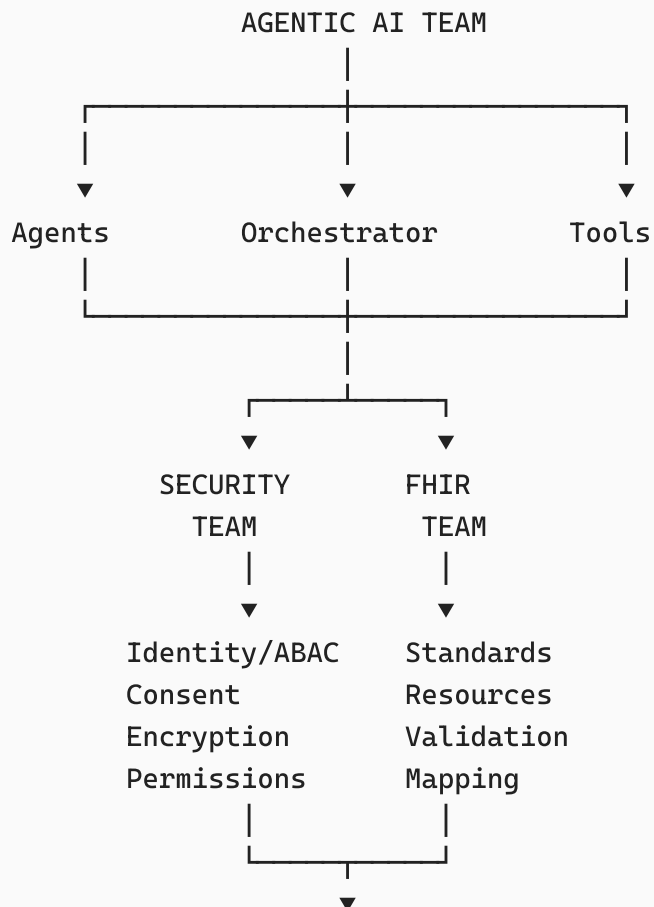


* FHIR Agent → coordinated with FHIR Team / future phase

Core Principle

The LLM reasons, the orchestrator coordinates, agents specialize, tools perform actions, governance controls the output, humans approve consequential clinical decisions, and VCAP records what happened.

Team Ownership



Scope of this proposal

Primary Agentic AI work:

- Multi-agent architecture
- Orchestration
- Agent roles
- Agent state
- Tool layer
- AI governance
- Human-in-the-loop
- AI provenance integration
- Agent testing

Dependencies / delegated work:

- Authentication → Security Team
- ABAC/consent → Security Team
- Encryption → Security Team
- FHIR implementation → FHIR/EHR Team
- Chaincode modifications → Blockchain Team
- Dashboard UI → Dashboard Team
- Large-scale infrastructure → Future Work

Future work, not initial scope:

- 1M+ record scalability
- distributed agent workers
- multimodal medical imaging
- advanced long-term memory
- autonomous pharmacy operations
- production-scale queues
- model routing/cost optimization
- physical machine-per-agent deployment